

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

TEMA:

"ENTREGABLE 3 · MI PRIMER DASHBOARD

SOLUCIÓN ANALÍTICA COMPLETA CON POSTGRESQL, PYTHON, STREAMLIT E INTELIGENCIA ARTIFICIAL

ANÁLISIS DE VENTAS, ENTREGAS Y SATISFACCIÓN DEL CLIENTE EN EL E-COMMERCE BRASILEÑO OLIST (2017–2018)"

ASIGNATURA:

INTELIGENCIA DE NEGOCIOS

CURSO Y PARALELO:

6/1

DOCENTE:

ING. ANTHONY PACHAY ESPINOZA, MSC.

ESTUDIANTE:

MARIA DE LOS ANGELES LLERENA HERNANDEZ

REPOSITORIO:

github.com/MariaLlerenaHernandez/Brazilian-E-Commerce-Public-Dataset-by-Olist

2026 – 2027

ÍNDICE

1. Introducción y alcance ... 3
1.1 Control de cambios respecto a los Entregables 1 y 2 ... 3
2. Dataset y comprensión del problema ... 4
2.1 Ficha del dataset ... 4
2.2 Tablas de origen y qué representa una fila ... 4
2.3 Variables principales ... 5
2.4 Problemas de calidad identificados ... 5
2.5 Justificación de la selección ... 5
2.6 Problema o necesidad de análisis ... 6
2.7 Usuarios potenciales y decisiones ... 6
3. Preguntas de negocio e indicadores ... 6
3.1 Preguntas de negocio ... 6
3.2 KPI definidos sobre el modelo dimensional ... 6
3.3 Conciliación con los valores preliminares del Entregable 1 ... 7
4. Limpieza y transformación de datos ... 7
5. Modelo dimensional ... 9
5.1 Granularidad ... 9
5.2 Tabla de hechos: fact_ventas ... 9
5.3 Dimensiones ... 10
5.4 Decisiones de diseño ... 10
6. Implementación en PostgreSQL ... 10
7. Validación de los datos ... 11
8. Capa analítica: vista y funciones SQL ... 12
9. Mockup de la aplicación ... 13
10. Matriz de trazabilidad ... 14
11. Aplicación Python + Streamlit ... 15
11.1 Estructura ... 15
11.2 Conexión entre Python y PostgreSQL ... 16
11.3 Cumplimiento de los requisitos de la aplicación ... 16
12. Resultados y análisis ... 16
12.1 P1 · Evolución de las ventas, categorías y estados ... 16
12.2 P2 · Puntualidad de las entregas ... 17
12.3 P3 · Efecto del retraso en la satisfacción ... 18
12.4 P4 · Medios de pago y cuotas ... 19
12.5 Decisiones que apoya la solución ... 19
13. Uso de Inteligencia Artificial ... 19
14. Conclusiones y limitaciones ... 20
14.1 Conclusiones ... 20

14.2 Limitaciones 20
15. Entrega, ejecución y video 20
Anexo A. Evidencias de ejecución 22
Referencias 26

1. Introducción y alcance

El presente documento consolida la versión final del proyecto de Inteligencia de Negocios desarrollado durante el corte. Integra, corrige y amplía los componentes trabajados en el Entregable 1 (comprensión del dataset y propuesta de modelo) y en el Entregable 2 (construcción del Data Mart), y añade la aplicación analítica en Streamlit, de modo que el entregable es **autocontenido**: no requiere consultar los documentos anteriores.

Objetivo. Diseñar e implementar una solución de Business Intelligence de extremo a extremo sobre el dataset público de Olist, que transforme ≈ 1,55 millones de filas de datos transaccionales en indicadores que apoyen decisiones comerciales, logísticas y de servicio al cliente.

Pregunta central del proyecto. La exploración del Entregable 1 sugirió que la puntualidad de la entrega determina la satisfacción del cliente. La solución final confirma y cuantifica esa relación, e identifica en qué meses, estados y vendedores se concentra el problema.

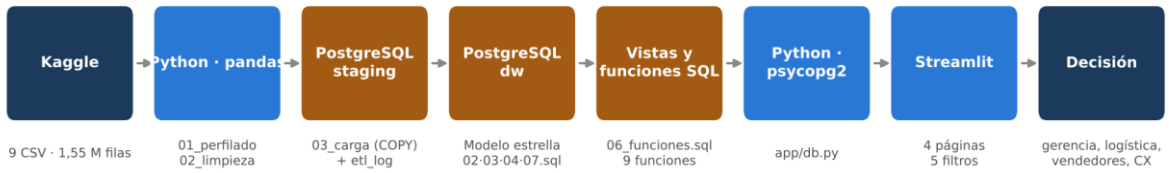


Figura 1. Arquitectura implementada: del CSV de Kaggle a la decisión de negocio.

1.1 Control de cambios respecto a los Entregables 1 y 2

La Tabla 1 resume las correcciones y mejoras incorporadas. Ninguna altera el grano ni la estructura del modelo estrella del Entregable 2; las cifras estructurales (112 279 filas de hechos, 97 905 pedidos válidos, R$ 13 449 529,68 de ingreso) se reproducen exactamente.

#	Componente	Situación en E1/E2	Cambio en el Entregable 3
1	Limpieza y transformación	Normalización realizada con funciones SQL (`fn_normalizar_ciudad`) sobre staging crudo.	Limpieza completa en Python/pandas (`etl/02_limpieza.py`), según la arquitectura exigida. Staging recibe datos ya limpios y con tipos correctos.
2	Trazabilidad de la carga	Sin registro de filas descartadas.	Bitácora `staging.etl_log` (filas origen, limpias, descartadas y acciones) y validación 0 «origen vs. staging».
3	Ciudades inválidas	Valores como `04482255`, correos o `sp` quedaban como ciudad.	Imputación por prefijo postal con la tabla de geolocalización deduplicada (8 casos).
4	Proceso de carga	Scripts SQL ejecutados uno a uno desde PowerShell.	`etl/03_carga.py`: COPY + scripts del DW + validaciones en **una sola transacción**; `run_pipeline.py` ejecuta todo.
5	Capa analítica	Vista + 4 funciones con 4 filtros, condición repetida en cada función.	Función base `fn_ventas_filtradas` (filtro definido una sola vez), 5 filtros (se añaden estado UF y medio de pago), 9 funciones y 8 indicadores en `fn_kpis`.
6	Ranking de vendedores	Solo orden por ingreso; llamada posicional `(NULL,NULL,NULL,NULL,5)`.	Orden por ingreso o por % de retraso y umbral de pedidos mínimos. Se invoca con parámetros con nombre: `fn_ranking_vendedores(p_limite => 5)`.

#	Componente	Situación en E1/E2	Cambio en el Entregable 3
7	Mockup	Una página HTML estática.	Cuatro páginas; cada componente indica la función SQL que lo alimenta (sección 9).
8	KPI del E1	Valores preliminares sin limpieza (p. ej. OTD 91,9 %).	Valores recalculados sobre el modelo y diferencias explicadas (sección 3.3).

Tabla 1. Control de cambios del proyecto.

2. Dataset y comprensión del problema

2.1 Ficha del dataset

Campo	Detalle
Nombre	Brazilian E-Commerce Public Dataset by Olist
Enlace de origen	https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
Publicado por	Olist, marketplace brasileño que conecta pequeños comercios con grandes tiendas en línea. Licencia CC BY-NC-SA 4.0.
Descripción general	Datos comerciales reales y anonimizados de pedidos realizados en Brasil a través de Olist. Cubre el ciclo completo de una compra: cliente, pedido, productos, vendedor, pago, envío y reseña.
Estructura	9 archivos CSV relacionados por las llaves `order_id`, `customer_id`, `product_id`, `seller_id` y el prefijo de código postal.
Cantidad de registros	99 441 pedidos · 112 650 ítems · 103 886 pagos · 99 224 reseñas · 99 441 clientes · 3 095 vendedores · 32 951 productos · 1 000 163 filas de geolocalización · 71 traducciones. Total ≈ 1,55 millones de filas.
Cantidad de columnas	52 columnas entre las 9 tablas (entre 2 y 9 por tabla).
Período temporal	Compras del 04/09/2016 al 17/10/2018. Período analizado (meses completos): **01/01/2017 – 31/08/2018 (20 meses)**.

Tabla 2. Ficha técnica del dataset.

2.2 Tablas de origen y qué representa una fila

Archivo CSV	Filas	Col.	Una fila representa…
olist_orders_dataset	99 441	8	Un pedido de un cliente, con su estado y las 5 fechas del ciclo (compra, aprobación, envío, entrega, entrega estimada).
olist_order_items_dataset	112 650	7	Un ítem de producto dentro de un pedido (`order_item_id` = 1, 2, 3…). El 90 % de los pedidos tiene un solo ítem; el máximo es 21.
olist_order_payments_dataset	103 886	5	Un pago aplicado a un pedido; un pedido puede combinar métodos (tarjeta + voucher).
olist_order_reviews_dataset	99 224	7	La reseña (1–5) que el cliente dejó sobre un pedido.
olist_customers_dataset	99 441	5	Un cliente asociado a un pedido. `customer_id` cambia en cada compra; la persona real es `customer_unique_id`.
olist_sellers_dataset	3 095	4	Un vendedor, con su código postal, ciudad y estado.
olist_products_dataset	32 951	9	Un producto, con categoría (portugués), medidas, peso y número de fotos.
olist_geolocation_dataset	1 000 163	5	Una coordenada (lat/long) asociada a un prefijo postal; hay varias por prefijo.
product_category_name_translation	71	2	La traducción de una categoría del portugués al inglés.

Tabla 3. Archivos del dataset y significado de una fila.

Fila base del proyecto: una línea de producto vendida dentro de un pedido (order item), enriquecida con los datos del pedido, cliente, producto, vendedor, pago y reseña. Es el nivel de detalle más fino disponible para analizar ventas y determina el grano de la tabla de hechos.

2.3 Variables principales

Tipo	Variables	Observaciones del perfilado
Identificadores	order_id, customer_id, customer_unique_id, product_id, seller_id, review_id, *_zip_code_prefix	Hash de 32 caracteres; sirven para unir tablas. El código postal es un identificador aunque llegue como número.
Numéricas	price, freight_value, payment_value, payment_installments, product_weight_g, medidas del producto	Precio: mediana R$ 74,99, máximo R$ 6 735 (distribución sesgada). Flete promedio R$ 20,03. Cuotas: 0 a 24.
Categóricas	product_category_name (73), payment_type (5), customer_state / seller_state (27), ciudades	Tarjeta de crédito 74 % de los pagos, boleto 19 %. São Paulo concentra el 42 % de los clientes.
Temporales	order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date	Llegan como texto. Permiten derivar días de entrega y días de retraso.
Estados / resultados	order_status (8 valores), review_score (1–5)	96 478 pedidos `delivered` (97 %). 58 % de reseñas con 5 estrellas y 11,5 % con 1 estrella.

Tabla 4. Variables principales y hallazgos del perfilado (`reports/perfilado.md`).

2.4 Problemas de calidad identificados

El perfilado automatizado (`etl/01_perfilado.py`) confirmó los hallazgos del Entregable 1:

- **Nulos:** 160 pedidos sin fecha de aprobación, 1 783 sin fecha de envío y 2 965 sin fecha de entrega; 8 pedidos `delivered` sin fecha de entrega; 610 productos sin categoría; 88 % de reseñas sin título y 59 % sin comentario.
- **Duplicados:** 261 831 filas idénticas en geolocalización (26 %); 814 `review_id` repetidos y 551 pedidos con más de una reseña.
- **Tipos incorrectos:** las 8 columnas de fecha llegan como texto; los códigos postales como número (pierden los ceros a la izquierda); medidas de producto como decimales.
- **Valores atípicos:** 8 427 ítems sobre el límite IQR de precio (R$ 277,40); 383 ítems con flete 0; 2 pagos con 0 cuotas; 4 productos con peso 0 g; 1 359 pedidos entregados a la transportista antes de ser aprobados.
- **Categorías inconsistentes:** la misma ciudad escrita de varias formas (`são paulo`, `sao paulo - sp`, `sao paulo / sao paulo`); ciudades inválidas en vendedores (`04482255`, un correo electrónico, `sp`); 2 categorías sin traducción; columnas mal escritas (`product_name_lenght`).
- **Meses incompletos:** sep-2016 (4 pedidos), oct-2016 (324), nov-2016 (0), dic-2016 (1), sep-2018 (16) y oct-2018 (4).
- **Integridad entre tablas:** 775 pedidos sin ítems (739 dentro del período analizado) y 1 pedido sin pago (de 2016, fuera del período).

2.5 Justificación de la selección

El dataset es multitabla y relacional, por lo que obliga a integrar fuentes como en un proceso ETL real; tiene un proceso de negocio claro (la venta) con dimensiones evidentes; permite analizar cuatro áreas del negocio (ventas, logística, satisfacción y pagos); ofrece casi 100 000 pedidos en 20 meses consistentes, suficientes para observar tendencia y estacionalidad; y presenta problemas de calidad reales que justifican la etapa de limpieza.

2.6 Problema o necesidad de análisis

Olist necesita conocer **cuánto vende, dónde y en qué categorías**, **qué tan puntuales son sus entregas** y **cuánto le cuesta la impuntualidad en satisfacción del cliente**, para dirigir la inversión comercial y corregir la operación logística donde más impacto tiene. Hoy esa información está dispersa en 9 archivos sin un modelo común ni indicadores definidos.

2.7 Usuarios potenciales y decisiones

Usuario	Qué necesita ver	Decisiones que apoya la solución
Gerencia general / dirección comercial	Ingreso, crecimiento, ticket, categorías y estados líderes.	En qué categorías invertir; reforzar campañas en meses pico; priorizar regiones con potencial.
Jefe de logística / operaciones	% de entregas a tiempo, tiempo de entrega y retrasos por estado y mes.	Renegociar o cambiar transportistas en estados críticos; reforzar capacidad antes de los picos; ajustar las fechas estimadas.
Gestión de vendedores	Ventas, % de retraso y CSAT por vendedor.	Capacitar, advertir o suspender vendedores con muchos retrasos; reconocer a los mejores.
Servicio al cliente / experiencia	CSAT, % de reseñas negativas y su relación con el retraso.	Contactar proactivamente a clientes con pedidos retrasados; ofrecer cupones de compensación.
Finanzas	Medios de pago, cuotas y ticket asociado.	Definir promociones de cuotas sin interés o descuentos por pago con boleto.

Tabla 5. Usuarios de la información y decisiones.

3. Preguntas de negocio e indicadores

3.1 Preguntas de negocio

1. **P1.** ¿Cómo evolucionan las ventas en el tiempo y qué categorías de productos y estados de Brasil generan más ingresos?
2. **P2.** ¿Qué porcentaje de pedidos se entrega a tiempo y en qué estados o vendedores se concentran los retrasos?
3. **P3.** ¿Influyen los retrasos en la entrega en la satisfacción del cliente (puntuación de las reseñas)?
4. **P4 (complementaria).** ¿Qué medios de pago y cuántas cuotas prefieren los clientes según el valor del pedido?

3.2 KPI definidos sobre el modelo dimensional

Todos los KPI se calculan sobre `dw.fact_ventas` y sus dimensiones mediante la función `dw.fn_kpis()`, excluyendo los pedidos `canceled` y `unavailable`. Los indicadores de nivel pedido se calculan sobre una fila por pedido para no sobrecontar los pedidos con varios ítems.

KPI	Fórmula sobre el modelo	Preg.	Valor final*
1. Ingreso total (GMV)	SUM(precio)	P1	R$ 13 449 529,68

KPI	Fórmula sobre el modelo	Preg.	Valor final*
2. Ticket promedio	SUM(valor_total) / COUNT(DISTINCT order_id)	P1	R$ 160,19
3. % entregas a tiempo (OTD)	100 × AVG(flag_a_tiempo), una fila por pedido	P2	93,2 %
4. Tiempo promedio de entrega	AVG(dias_entrega), una fila por pedido	P2	12,5 días
5. Satisfacción promedio (CSAT)	AVG(review_score), una fila por pedido	P3	4,12 / 5
6. % reseñas negativas	100 × COUNT(review_score ≤ 2) / COUNT(review_score)	P3	13,8 %
Contexto: pedidos válidos	COUNT(DISTINCT order_id)	P1	97 905
Contexto: flete total	SUM(valor_flete)	P1	R$ 2 234 177,06

*Tabla 6. KPI implementados. *Sin filtros: 01/01/2017–31/08/2018, pedidos no cancelados. Fuente:* `SELECT * FROM dw.fn_kpis();`

3.3 Conciliación con los valores preliminares del Entregable 1

El Entregable 1 presentó valores exploratorios sin limpieza. La Tabla 7 explica cada diferencia; ninguna se debe a un error de carga, sino a reglas de cálculo explícitas:

Indicador	E1 (preliminar)	E3 (final)	Explicación de la diferencia
Ingreso en productos	≈ R$ 13,6 M	R$ 13,45 M	E3 limita a ene-2017–ago-2018 y excluye pedidos cancelados/no disponibles (R$ 13 541 712,78 antes de excluirlos).
Ticket promedio	R$ 160,99	R$ 160,19	Mismo criterio de período y exclusión de cancelados.
% entregas a tiempo	≈ 91,9 %	93,2 %	E1 comparaba fecha y hora; la fecha estimada no tiene hora (00:00:00), por lo que 1 291 pedidos entregados el mismo día estimado se contaban como retrasados. E3 compara fechas de calendario.
CSAT con retraso	2,57	2,27	Mismo efecto: los 1 291 pedidos «del mismo día», con reseñas altas, ya no se mezclan con los retrasados reales.
Tiempo de entrega	12,1 días	12,5 días	E3 cuenta días de calendario y solo pedidos `delivered` del período.
Tasa de cancelación	≈ 1,2 %	No se usa como KPI	El 62 % de los pedidos cancelados o no disponibles del período (734 de 1 182) no tiene ítems y no puede existir en una tabla de hechos con grano de ítem. Se sustituye por el % de reseñas negativas y el tiempo de entrega; los pedidos sin ítems se controlan en la validación 2c.

Tabla 7. Conciliación de indicadores E1 → E3.

4. Limpieza y transformación de datos

La limpieza se implementó en Python con pandas en `etl/02_limpieza.py`. Cada tratamiento responde a un problema identificado en la sección 2.4. El script escribe un CSV limpio por tabla en `data/clean/` y la bitácora `reports/bitacora_limpieza.csv`, que se carga en `staging.etl_log`.

Tratamiento	Problema	Acción aplicada
Valores nulos	Fechas faltantes; productos sin categoría; peso 0.	Se conservan los nulos lógicos de fechas (pedidos no entregados). Categoría nula → `sin_categoria`. Peso 0 g → nulo. Medidas de entrega solo para pedidos `delivered`.

Tratamiento	Problema	Acción aplicada
Duplicados	Reseñas múltiples por pedido; geolocalización repetida.	Una reseña por pedido: la más reciente (`review_creation_date`, `review_answer_timestamp`). Geolocalización deduplicada y resumida a 19 015 prefijos postales.
Tipos de datos	Fechas como texto; códigos postales numéricos; medidas decimales.	`pd.to_datetime` en las 8 fechas; código postal a texto de 5 dígitos con `zfill(5)`; medidas a entero (`Int64`).
Estandarización de fechas	Fecha estimada sin hora.	En la tabla de hechos todas las fechas se comparan como DATE (sección 3.3).
Estandarización de categorías	Ciudades en múltiples formas; estado y tipos de pago con mayúsculas variables.	Ciudades: minúsculas, sin tildes, sin sufijos (`- sp`, `/`, `,`, sigla final). Ciudades inválidas imputadas por prefijo postal. 2 traducciones de categoría completadas.
Valores fuera de rango	Precios altos; flete 0; 0 cuotas; fechas incoherentes.	Precios altos y flete 0 se conservan (ventas reales). 0 cuotas → 1. Bandera `flag_fechas_incoherentes` para 1 359 pedidos (se marcan, no se eliminan). Restricciones CHECK en la base.
Columnas irrelevantes	Texto libre y metadatos sin uso analítico.	Se eliminan títulos y comentarios de reseñas, `shipping_limit_date`, `product_name_lenght`, `product_description_lenght`, `product_photos_qty`.
Variables derivadas	Medidas necesarias para los KPI.	`valor_total`, `dias_entrega`, `dias_retraso`, `flag_a_tiempo`, `volumen_cm3`, `region`, `es_fin_semana`, rango de cuotas y pago principal.

Tabla 8. Tratamientos de limpieza y transformación.

Bitácora del ETL (coherencia origen → carga). La Tabla 9 muestra las filas de cada archivo original y las cargadas en staging, generadas automáticamente en cada ejecución:

Tabla	Filas origen	Filas limpias	Descartadas	Motivo
orders	99 441	99 441	0	Solo cambios de tipo y bandera de fechas.
order_items	112 650	112 650	0	Sin duplicados por (order_id, order_item_id).
order_payments	103 886	103 886	0	0 cuotas → 1.
order_reviews	99 224	98 673	551	Se conserva una reseña por pedido (la más reciente).
customers	99 441	99 441	0	Normalización de ciudad y código postal.
sellers	3 095	3 095	0	611 → 581 ciudades distintas; 7 imputadas.
products	32 951	32 951	0	Categoría, medidas y peso.
category_translation	71	74	+3	2 traducciones faltantes y `sin_categoria`.
geolocation	1 000 163	0 (no se carga)	—	Se usa solo para imputar ciudades inválidas.

Tabla 9. Bitácora `staging.etl_log`. Los valores resaltados dependen de la versión del archivo de reseñas descargada de Kaggle; se confirman en `reports/validaciones.md` tras ejecutar el pipeline.

Extracto de etl/02_limpieza.py · normalización de ciudades

```
def normalizar_ciudad(serie: pd.Series) -> pd.Series:
    """'São Paulo - SP', 'sao paulo / sao paulo', ' SAO  PAULO ' -> 'sao paulo'."""
    s = serie.fillna("").astype(str).str.lower().str.replace("´", "'").str.replace("`", "'")
    s = s.map(sin_tildes)
    s = s.str.split(r"\s+-\s+|/|\\|,", n=1, regex=True).str[0]  # quita sufijos de estado
    s = s.str.replace(r"[^a-z' ]", " ", regex=True)              # deja solo letras
```

```
    s = s.str.replace(r"\s+", " ", regex=True).str.strip()
    s = s.str.replace(r"(?<=\w)
(ac|al|ap|am|ba|ce|df|es|go|ma|mt|ms|mg|pa|pb|pr|pe|pi|rj|rn|rs|ro|rr|sc|sp|se|to)$",
                      "", regex=True)                              # 'sao paulo sp' -> 'sao paulo'
    return s.replace("", np.nan)
```

5. Modelo dimensional

Se implementó un **modelo estrella** centrado en el proceso de negocio **venta**, con una tabla de hechos y seis dimensiones en el esquema `dw` de PostgreSQL 16 (Figura 2).

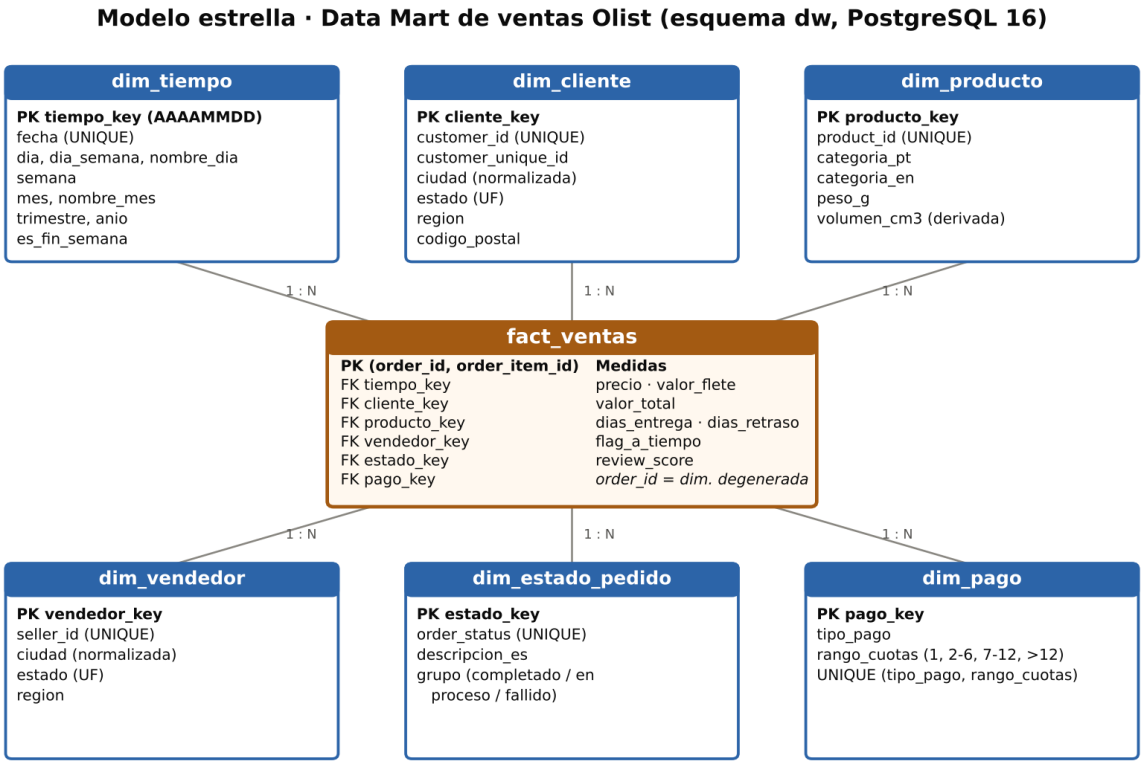


Figura 2. Diagrama estrella del Data Mart (esquema dw).

5.1 Granularidad

> **Una fila de la tabla de hechos representa un ítem de producto vendido dentro de un pedido** (`order_id` + `order_item_id`), comprado entre el 01/01/2017 y el 31/08/2018.

Se eligió el nivel más fino disponible para poder analizar por producto, categoría y vendedor sin perder detalle; los indicadores de nivel pedido (OTD, CSAT, ticket) se obtienen agregando a una fila por pedido. La tabla contiene **112 279 filas**.

5.2 Tabla de hechos: fact_ventas

Columna	Tipo	Rol	Descripción / agregación
order_id, order_item_id	VARCHAR(32), SMALLINT	PK compuesta	`order_id` es dimensión degenerada: se usa para contar pedidos distintos.
tiempo_key … pago_key	INTEGER	6 FK	Referencias a las 6 dimensiones (NOT NULL).
precio	NUMERIC(10,2)	Medida	Precio del producto (R$). Suma.
valor_flete	NUMERIC(10,2)	Medida	Flete asignado al ítem. Suma.

Columna	Tipo	Rol	Descripción / agregación
valor_total	NUMERIC(10,2)	Medida derivada	precio + valor_flete. Suma.
dias_entrega	INTEGER	Medida derivada	Fecha de entrega − fecha de compra. Promedio.
dias_retraso	INTEGER	Medida derivada	Fecha de entrega − fecha estimada (negativo = adelantado). Promedio / tramos.
flag_a_tiempo	SMALLINT	Medida derivada	1 si fecha de entrega ≤ fecha estimada; 0 si no. Promedio = % OTD.
review_score	SMALLINT	Medida	Puntuación 1–5 de la reseña del pedido. Promedio por pedido.

Tabla 10. Estructura de la tabla de hechos.

5.3 Dimensiones

Dimensión	Filas	Clave primaria	Atributos	Origen
dim_tiempo	730	tiempo_key (AAAAMMDD)	fecha, día, día de la semana, nombre del día, semana, mes, nombre del mes, trimestre, año, fin de semana	Generada con `generate_series` (2017–2018)
dim_cliente	99 441	cliente_key (SERIAL)	customer_id (UNIQUE), customer_unique_id, ciudad, estado, región, código postal	customers
dim_producto	32 951	producto_key (SERIAL)	product_id (UNIQUE), categoría PT y EN, peso, volumen	products + translation
dim_vendedor	3 095	vendedor_key (SERIAL)	seller_id (UNIQUE), ciudad, estado, región	sellers
dim_estado_pedido	8	estado_key (SERIAL)	order_status (UNIQUE), descripción en español, grupo (completado / en proceso / fallido)	orders.order_status
dim_pago	20	pago_key (SERIAL)	tipo de pago × rango de cuotas (1, 2-6, 7-12, >12); UNIQUE (tipo, rango)	order_payments (pago principal)

Tabla 11. Dimensiones del modelo.

5.4 Decisiones de diseño

- **Pago principal.** Un pedido puede tener varios pagos; para mantener la relación 1:N con la tabla de hechos se asigna el pago de **mayor valor** (en empate, el primero por `payment_sequential`). Se descartó una segunda tabla de hechos de pagos porque la P4 solo requiere el medio de pago predominante.
- **Reseña por pedido.** Se conserva una reseña por pedido y se repite en sus ítems; los promedios se calculan sobre pedidos distintos (`DISTINCT ON (order_id)`).
- **Medidas de entrega.** Solo se calculan para pedidos `delivered`; un pedido cancelado con fecha de entrega no constituye una entrega válida.
- **Región.** La región de Brasil se deriva del estado (UF) con la función `dw.fn_region()`, lo que permite un filtro jerárquico región → estado.

6. Implementación en PostgreSQL

La base `bi_database` corre en PostgreSQL 16 dentro de Docker (`docker-compose.yml`, contenedor `bi-postgres`, con Adminer en el puerto 8080). El proceso completo se ejecuta con `python etl/run_pipeline.py`.

Orden	Script	Contenido
1	etl/01_perfilado.py	Perfilamiento de los 9 CSV → `reports/perfilado.md`.
2	etl/02_limpieza.py	Limpieza con pandas → `data/clean/*.csv` + bitácora.
3	sql/01_staging.sql	Esquema `staging` con tipos correctos y tabla `etl_log`.
4	etl/03_carga.py	COPY de los CSV limpios a staging y ejecución de los scripts siguientes en una transacción.
5	sql/02_dw_tablas.sql	Esquema `dw`: 6 dimensiones y `fact_ventas` con PK, FK e índices sobre las FK.
6	sql/03_dimensiones.sql	Carga de dimensiones y función `dw.fn_region`.
7	sql/04_fact_ventas.sql	Carga de la tabla de hechos con medidas derivadas.
8	sql/07_restricciones.sql	15 restricciones CHECK de calidad.
9	sql/06_funciones.sql	Vista `dw.vw_ventas_detalle` y funciones de consumo analítico.
10	sql/05_validaciones.sql	14 consultas de validación → `reports/validaciones.md`.

Tabla 12. Scripts y orden de ejecución.

Extracto de sql/04_fact_ventas.sql · selección del pago principal

```
WITH pago_principal AS (
    -- Un pedido puede tener varios pagos (tarjeta + voucher). Se toma como
    -- pago principal el de MAYOR valor; en caso de empate, el primero.
    SELECT DISTINCT ON (order_id)
           order_id,
           payment_type,
           CASE WHEN payment_installments <= 1  THEN '1'
                WHEN payment_installments <= 6  THEN '2-6'
                WHEN payment_installments <= 12 THEN '7-12'
                ELSE '>12' END AS rango_cuotas
    FROM staging.order_payments
    ORDER BY order_id, payment_value DESC, payment_sequential
```

Extracto de sql/04_fact_ventas.sql · medidas derivadas

```
       b.price + b.freight_value,                                  -- valor_total
       b.f_entrega - b.f_compra,                                   -- dias_entrega (NULL si no se
entregó)
       b.f_entrega - b.f_estimada,                                 -- dias_retraso (negativo =
adelantado)
       CASE WHEN b.f_entrega IS NULL THEN NULL
            WHEN b.f_entrega <= b.f_estimada THEN 1 ELSE 0 END, -- flag_a_tiempo
       b.review_score
```

Integridad referencial y reglas de calidad. Las 6 FK de `fact_ventas` son NOT NULL y referencian las PK de las dimensiones. El script `07_restricciones.sql` añade restricciones CHECK, por ejemplo: `precio >= 0`, `valor_total = precio + valor_flete`, `review_score BETWEEN 1 AND 5`, `flag_a_tiempo IN (0, 1)`, `dias_entrega >= 0`, estado con patrón `^[A-Z]{2}$` y código postal `^[0-9]{5}$`. Un intento de `UPDATE dw.fact_ventas SET review_score = 9` es rechazado por la base con el error `violates check constraint "ck_fact_review"`.

7. Validación de los datos

Las consultas de `sql/05_validaciones.sql` se ejecutan automáticamente al final de la carga. La Tabla 13 resume el resultado obtenido frente al esperado:

#	Verificación	Esperado	Obtenido	Estado
0	Coherencia origen → staging (8 tablas)	filas limpias = filas en staging	8 de 8 coinciden	OK

#	Verificación	Esperado	Obtenido	Estado
1	Registros por tabla	730 / 99 441 / 32 951 / 3 095 / 8 / 20	730 / 99 441 / 32 951 / 3 095 / 8 / 20	OK
1b	Ítems del período en staging vs. filas de hechos	Iguales	112 279 = 112 279	OK
2	Filas de hechos sin dimensión (6 FK)	0	0 en las 6 claves	OK
2b	Duplicados en el grano (order_id, order_item_id)	0	0	OK
2c	Pedidos del período sin ítems	Solo estados fallidos	739 (canceled, created, unavailable)	OK
3	Nulos en precio / flete	0	0	OK
3a	Nulos en medidas de entrega	Explicados por el estado	2 407: shipped 1 174, canceled 527, processing 355, invoiced 340, delivered 8, approved 3	OK
3	Ítems sin reseña	Pedidos sin reseña	936	OK
3b	Nulos en dimensiones	Solo los documentados	610 sin categoría, 6 sin peso, 0 clientes y 0 vendedores sin ciudad	OK
4	Totales del período (incl. cancelados)	Igual al E2	112 279 ítems · 98 353 pedidos · R$ 13 541 712,78 · flete R$ 2 244 490,79 · ticket R$ 160,51	OK
4a	Totales staging vs. hechos	Diferencia 0,00	0,00 en ingreso y en flete	OK
4b	valor_total ≠ precio + flete	0	0	OK
4c	KPI recalculados sobre la tabla de hechos	Igual a fn_kpis()	12,5 días · 93,2 % · CSAT 4,12	OK
5	fn_kpis vs. suma mensual vs. suma por estado	Iguales	R$ 13 449 529,68 en las tres	OK

Tabla 13. Resultados de validación (`reports/validaciones.md`). Las cifras estructurales coinciden exactamente con las reportadas en el Entregable 2.

8. Capa analítica: vista y funciones SQL

La aplicación no consulta las tablas directamente: consume una vista base y nueve funciones del esquema `dw`. Todas comparten la misma firma de filtros (`p_fecha_ini`, `p_fecha_fin`, `p_region`, `p_categoria`, `p_estado`, `p_tipo_pago`, donde NULL significa «todos») y se apoyan en `dw.fn_ventas_filtradas`, de modo que un filtro se define una sola vez y se aplica igual en todos los componentes.

Objeto SQL	Tipo	Devuelve	Preg.
dw.vw_ventas_detalle	Vista	Hechos + atributos de las 6 dimensiones	Base
dw.fn_ventas_filtradas	Función (SETOF vista)	Filas de la vista con los filtros aplicados y sin cancelados	Base
dw.fn_kpis	Función	8 indicadores en una fila	P1–P3
dw.fn_ingresos_mensuales	Función	Ingreso, pedidos, ticket y variación % por mes (función ventana LAG)	P1
dw.fn_ventas_por_categoria	Función	Ingreso, pedidos, participación y precio promedio por categoría (Top N)	P1
dw.fn_ventas_por_estado	Función	Ingreso, participación, OTD, días de entrega y CSAT por estado	P1, P2

Objeto SQL	Tipo	Devuelve	Preg.
dw.fn_csat_por_puntualidad	Función	CSAT y % de reseñas negativas a tiempo vs. con retraso	P3
dw.fn_csat_por_retraso	Función	CSAT y % negativas en 7 tramos de días de retraso	P3
dw.fn_medios_pago	Función	Pedidos, ingreso y ticket por medio de pago y rango de cuotas	P4
dw.fn_ranking_vendedores	Función	Pedidos, ingreso, % con retraso, días de entrega y CSAT por vendedor	P2

Tabla 14. Objetos SQL de consumo analítico (`sql/06_funciones.sql`).

Extracto de sql/06_funciones.sql · filtro común

```
CREATE FUNCTION dw.fn_ventas_filtradas(
    p_fecha_ini DATE DEFAULT '2017-01-01',
    p_fecha_fin DATE DEFAULT '2018-08-31',
    p_region    TEXT DEFAULT NULL,
    p_categoria TEXT DEFAULT NULL,
    p_estado    TEXT DEFAULT NULL,
    p_tipo_pago TEXT DEFAULT NULL)
RETURNS SETOF dw.vw_ventas_detalle
LANGUAGE sql STABLE AS $$
    SELECT *
    FROM dw.vw_ventas_detalle v
    WHERE NOT v.es_cancelado
      AND v.fecha BETWEEN COALESCE(p_fecha_ini, '1900-01-01') AND COALESCE(p_fecha_fin, '2999-12-31')
      AND (p_region    IS NULL OR v.region_cliente = p_region)
      AND (p_categoria IS NULL OR v.categoria      = p_categoria)
      AND (p_estado    IS NULL OR v.estado_cliente = p_estado)
      AND (p_tipo_pago IS NULL OR v.tipo_pago      = p_tipo_pago);
$$;
```

Extracto de sql/06_funciones.sql · satisfacción según puntualidad (P3)

```
CREATE FUNCTION dw.fn_csat_por_puntualidad(
    p_fecha_ini DATE DEFAULT '2017-01-01', p_fecha_fin DATE DEFAULT '2018-08-31',
    p_region TEXT DEFAULT NULL, p_categoria TEXT DEFAULT NULL,
    p_estado TEXT DEFAULT NULL, p_tipo_pago TEXT DEFAULT NULL)
RETURNS TABLE (grupo TEXT, pedidos BIGINT, csat_promedio NUMERIC, pct_resenas_negativas NUMERIC)
LANGUAGE sql STABLE AS $$
    WITH ped AS (
        SELECT DISTINCT ON (order_id) order_id, flag_a_tiempo, review_score
        FROM dw.fn_ventas_filtradas(p_fecha_ini, p_fecha_fin, p_region, p_categoria, p_estado,
p_tipo_pago)
        WHERE flag_a_tiempo IS NOT NULL
          AND review_score IS NOT NULL
        ORDER BY order_id
    )
    SELECT (CASE WHEN flag_a_tiempo = 1 THEN 'A tiempo' ELSE 'Con retraso' END)::text,
           count(*),
           round(avg(review_score), 2),
           round(100.0 * count(*) FILTER (WHERE review_score <= 2) / count(*), 1)
    FROM ped
    GROUP BY flag_a_tiempo
    ORDER BY flag_a_tiempo DESC;
$$;
```

9. Mockup de la aplicación

El mockup del Entregable 2 representaba una sola página. Se corrigió para reflejar la estructura definitiva de cuatro páginas, con una barra lateral de filtros común y la referencia a la función SQL que alimenta cada componente (Figura 3).

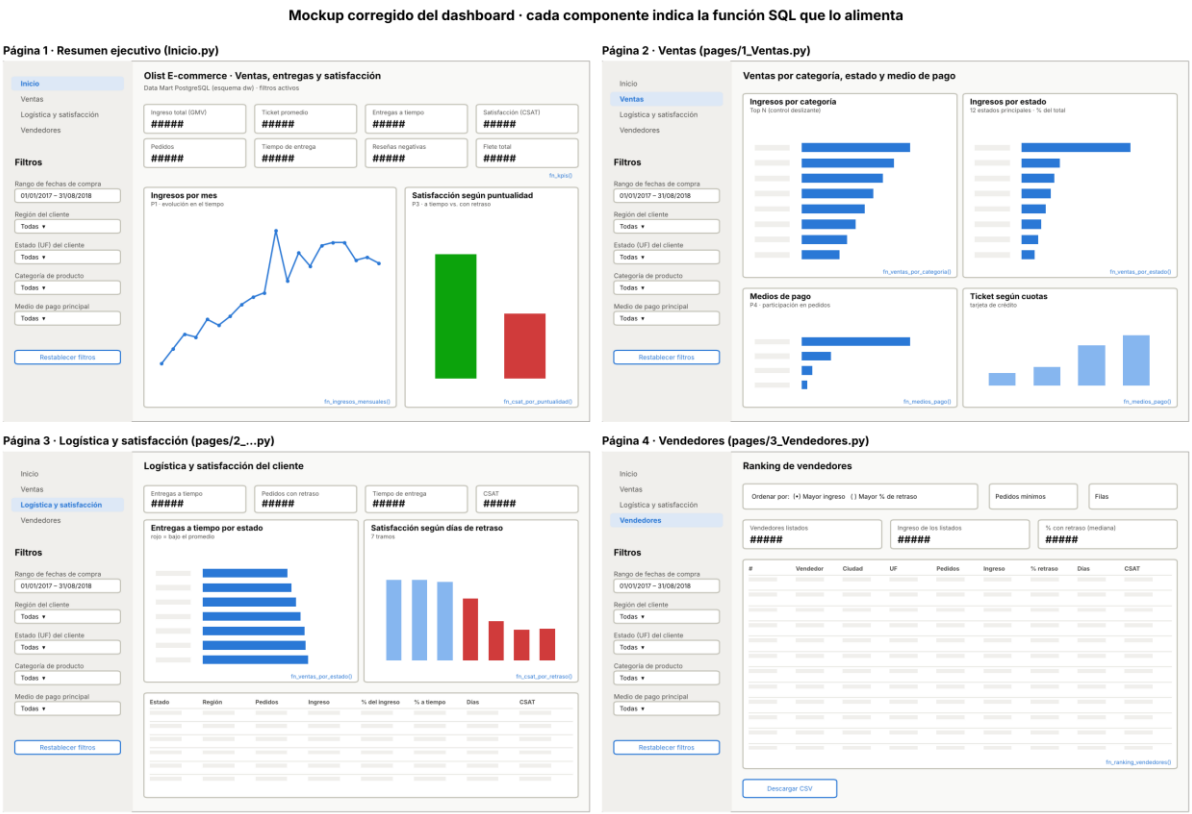


Figura 3. Mockup corregido de las cuatro páginas del dashboard.

Corrección	Motivo
De 1 a 4 páginas (Resumen, Ventas, Logística y satisfacción, Vendedores).	Cada página responde a una pregunta y a un usuario distinto; evita una pantalla saturada.
De 4 a 5 filtros, en barra lateral persistente; estado (UF) en cascada con la región.	El análisis logístico exige bajar de región a estado; los filtros deben mantenerse al navegar.
De 4 a 8 tarjetas KPI en la página de inicio.	Se incorporan tiempo de entrega y % de reseñas negativas, que sustituyen a la tasa de cancelación (sección 3.3).
Nuevo gráfico de satisfacción por tramos de días de retraso.	Mide la intensidad del efecto del retraso, no solo su existencia (P3).
Ranking de vendedores con orden por % de retraso y umbral de pedidos.	Ordenar solo por ingreso ocultaba a los vendedores problemáticos (P2).
Cada componente muestra «Ver consulta SQL».	Trazabilidad visible entre el gráfico y el dato.

Tabla 15. Correcciones del mockup respecto al Entregable 2.

10. Matriz de trazabilidad

Filtros globales aplicados a todos los componentes: **F1** rango de fechas · **F2** región · **F3** estado (UF) · **F4** categoría · **F5** medio de pago.

Componente (página)	Pregunta / KPI	Filtros	Datos utilizados	Vista / Función SQL
Tarjetas KPI (Inicio, Logística)	P1–P3 · KPI 1 a 6	F1–F5	precio, valor_total, valor_flete, flag_a_tiempo, dias_entrega, review_score, order_id	dw.fn_kpis()
Líneas: ingresos por mes (Inicio)	P1 · evolución y variación %	F1–F5	precio + dim_tiempo (año, mes)	dw.fn_ingresos_mensuales()

Componente (página)	Pregunta / KPI	Filtros	Datos utilizados	Vista / Función SQL
Barras: CSAT según puntualidad (Inicio)	P3 · KPI 5 y 6	F1–F5	flag_a_tiempo, review_score	dw.fn_csat_por_puntualidad()
Barras: ingresos por categoría (Ventas)	P1 · categorías líderes	F1–F5, Top N	precio + dim_producto.categoria_en	dw.fn_ventas_por_categoria()
Barras: ingresos por estado (Ventas)	P1 · concentración geográfica	F1–F5	precio + dim_cliente (estado, región)	dw.fn_ventas_por_estado()
Barras: medios de pago y ticket por cuotas (Ventas)	P4	F1–F5	dim_pago + valor_total	dw.fn_medios_pago()
Barras: OTD por estado (Logística)	P2 · KPI 3 y 4	F1–F5	flag_a_tiempo, dias_entrega + dim_cliente	dw.fn_ventas_por_estado()
Barras: CSAT por días de retraso (Logística)	P3 · KPI 5 y 6	F1–F5	dias_retraso, review_score	dw.fn_csat_por_retraso()
Tabla: detalle por estado (Logística)	P1, P2	F1–F5	Igual que OTD por estado	dw.fn_ventas_por_estado()
Tabla: ranking de vendedores (Vendedores)	P2 · retrasos por vendedor	F1–F5, orden, mínimo, filas	precio, flag_a_tiempo, dias_entrega, review_score + dim_vendedor	dw.fn_ranking_vendedores()

Tabla 16. Matriz de trazabilidad. Versión ampliada con la trazabilidad de cada filtro en `docs/matriz_trazabilidad.md`.

Filtro	Widget Streamlit	Parámetro SQL	Columna de la vista	Dimensión de origen
F1 Rango de fechas	st.date_input	p_fecha_ini, p_fecha_fin	fecha	dim_tiempo.fecha
F2 Región	st.selectbox	p_region	region_cliente	dim_cliente.region
F3 Estado (UF)	st.selectbox (cascada)	p_estado	estado_cliente	dim_cliente.estado
F4 Categoría	st.selectbox	p_categoria	categoria	dim_producto.categoria_en
F5 Medio de pago	st.selectbox	p_tipo_pago	tipo_pago	dim_pago.tipo_pago

Tabla 17. Trazabilidad de los filtros.

11. Aplicación Python + Streamlit

La aplicación (`app/`) es multipágina y se ejecuta con `streamlit run app/Inicio.py`. Toda la información proviene de PostgreSQL: el módulo `app/db.py` abre la conexión con psycopg2 y solo permite invocar las funciones de una lista blanca, con parámetros enlazados (sin concatenar valores del usuario en el SQL).

11.1 Estructura

Archivo	Página / función	Contenido
app/Inicio.py	Página 1 · Resumen ejecutivo	8 tarjetas KPI, línea de ingresos mensuales con el pico anotado, barras de CSAT a tiempo vs. con retraso.
app/pages/1_Ventas.py	Página 2 · Ventas	Top N de categorías (control deslizante), 12 estados principales, medios de pago y ticket por cuotas.
app/pages/2_Logistica_y_Satisfaccion.py	Página 3 · Logística y satisfacción	4 KPI logísticos, OTD por estado con referencia al promedio, CSAT por tramos de retraso y tabla de detalle por estado.

Archivo	Página / función	Contenido
app/pages/3_Vendedores.py	Página 4 · Vendedores	Ranking con orden por ingreso o % de retraso, pedidos mínimos, número de filas y descarga CSV.
app/componentes.py	Compartido	Barra lateral con los 5 filtros persistentes, formatos (R$, %), estilo de gráficos y expansor «Ver consulta SQL».
app/db.py	Acceso a datos	Conexión, lista blanca de funciones, caché de resultados (10 min) y conversión a DataFrame.

Tabla 18. Estructura de la aplicación.

11.2 Conexión entre Python y PostgreSQL

Extracto de app/db.py · llamada a las funciones SQL

```
@st.cache_data(ttl=600, show_spinner="Consultando PostgreSQL…")
def llamar(funcion: str, **params) -> pd.DataFrame:
    """Llama a una función del esquema dw usando notación con nombre:
    SELECT * FROM dw.fn_x(p_fecha_ini => %(p_fecha_ini)s, ...)"""
    if funcion not in FUNCIONES:
        raise ValueError(f"Función no permitida: {funcion}")
    args = ", ".join(f"{k} => %({k})s" for k in params)
    return consultar(f"SELECT * FROM dw.{funcion}({args})", params)
```

Cada página arma los parámetros a partir de los filtros y llama, por ejemplo, `llamar("fn_kpis", **p)`, lo que ejecuta `SELECT * FROM dw.fn_kpis(p_fecha_ini => …, p_region => …)`. Al cambiar un filtro, Streamlit vuelve a ejecutar la página y la consulta se repite con los nuevos valores; por eso los filtros **afectan realmente** a las consultas y no a datos precargados.

11.3 Cumplimiento de los requisitos de la aplicación

Requisito	Mínimo	Implementado
Conexión real con PostgreSQL	Sí	psycopg2 → funciones `dw.fn_*` (app/db.py)
Vistas, páginas o secciones analíticas	2	4 páginas
KPI	3	6 KPI + 2 indicadores de contexto
Filtros interactivos	3	5 globales (fechas, región, estado, categoría, medio de pago) + 4 locales
Gráficos	2	8 (línea, barras verticales y horizontales)
Tabla o vista de detalle	1	2 (detalle por estado y ranking de vendedores, con descarga CSV)
Consultas que responden a las preguntas	P1–P3	P1, P2, P3 y P4 (Tabla 16)
No leer el CSV original	Obligatorio	Cumplido: ningún archivo de `app/` lee CSV; las opciones de los filtros se leen de las dimensiones

Tabla 19. Requisitos de la aplicación.

12. Resultados y análisis

Los resultados se obtienen de las funciones SQL sin filtros (ene-2017 – ago-2018, pedidos no cancelados), salvo que se indique otra cosa.

12.1 P1 · Evolución de las ventas, categorías y estados

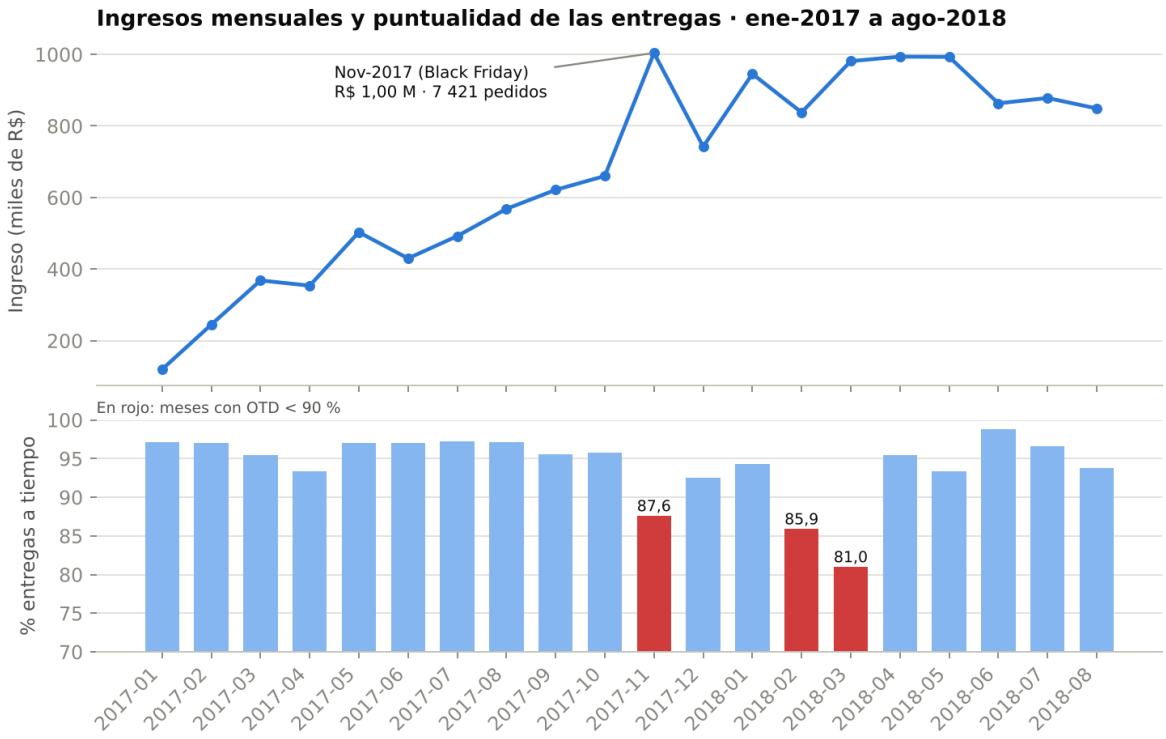


Figura 4. Ingresos mensuales (arriba) y % de entregas a tiempo por mes (abajo).

- **Crecimiento.** El ingreso de enero–agosto 2018 (R$ 7,34 M) supera en **138,3 %** al de enero–agosto 2017 (R$ 3,08 M). Desde marzo de 2018 el ingreso mensual se estabiliza entre R$ 0,85 M y R$ 1,0 M.
- **Estacionalidad.** Noviembre 2017 es el mes de mayor ingreso: **R$ 1 003 862,14** y 7 421 pedidos (+52,1 % frente a octubre), efecto del Black Friday.
- **Categorías.** Lideran health_beauty (9,3 % del ingreso), watches_gifts (8,9 %) y bed_bath_table (7,7 %); las 10 primeras concentran el 62,6 % del ingreso.
- **Estados.** São Paulo aporta el 38,3 % del ingreso, Río de Janeiro el 13,4 % y Minas Gerais el 11,7 % (63,4 % entre los tres); la región Sudeste suma el 65,4 %. En cambio, el ticket promedio es mayor en el Norte (R$ 223,32) y el Nordeste (R$ 201,17) que en el Sudeste (R$ 150,84).

12.2 P2 · Puntualidad de las entregas

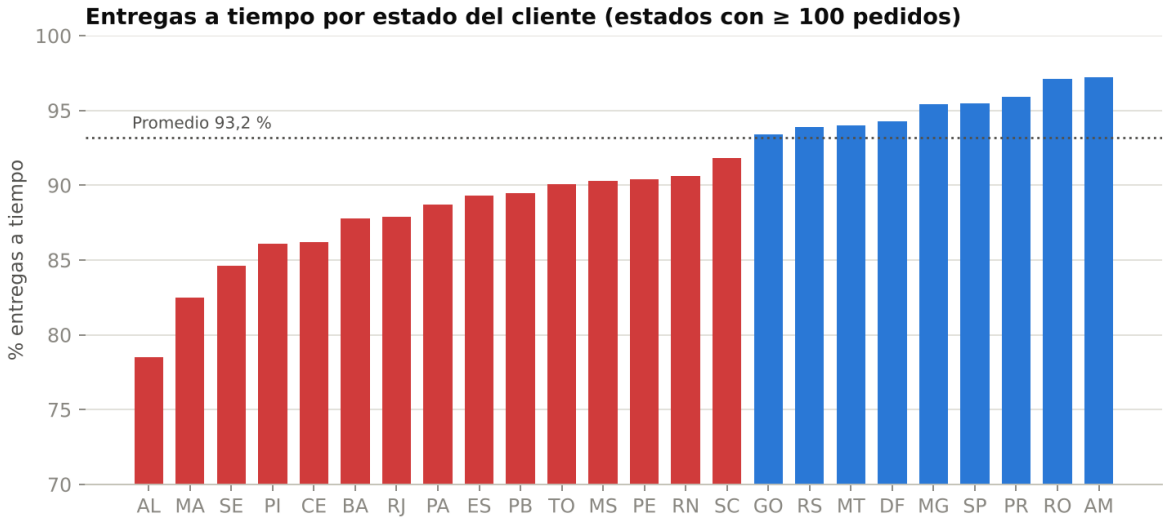


Figura 5. % de entregas a tiempo por estado del cliente (estados con al menos 100 pedidos).

- **Nivel general.** El 93,2 % de los pedidos se entrega a tiempo; los **6 531** pedidos con retraso llegan en promedio 10,6 días después de la fecha estimada.
- **Concentración geográfica.** Los 9 estados del Nordeste están por debajo del promedio: Alagoas (78,5 %), Maranhão (82,5 %) y Sergipe (84,6 %) son los peores. La región Nordeste promedia 87,2 % de puntualidad y 19,9 días de entrega, frente a 93,9 % y 10,7 días del Sudeste.
- **Río de Janeiro** tiene el 12,8 % de los pedidos pero genera el **22,9 % de los retrasos**: por volumen, es el estado donde una mejora logística tiene mayor impacto.
- **Picos de demanda.** La puntualidad cae a 87,6 % en noviembre 2017 y a 85,9 % y 81,0 % en febrero y marzo 2018 (Figura 4): la operación no escala al mismo ritmo que la demanda.
- **Vendedores.** De los 633 vendedores con al menos 30 pedidos, 38 superan el 15 % de pedidos con retraso; representan solo el 3,4 % del ingreso de ese grupo, por lo que pueden corregirse sin riesgo comercial relevante.

12.3 P3 · Efecto del retraso en la satisfacción



Figura 6. CSAT promedio según días de diferencia entre la entrega real y la estimada.

- Los pedidos entregados a tiempo obtienen un CSAT de **4,29**, frente a **2,27** de los retrasados (Entregable 2: 89 182 y 6 378 pedidos con reseña, respectivamente): una brecha de 2,02 puntos sobre 5.
- El efecto es inmediato y creciente: mientras el pedido llega antes o en la fecha estimada, el CSAT se mantiene entre 4,19 y 4,32; con 1 a 3 días de retraso cae a 3,29, con 4 a 7 días a 2,10 y con más de una semana se estabiliza alrededor de 1,7, con ≈ 80 % de reseñas negativas.
- Las regiones con peor puntualidad tienen también el menor CSAT (Nordeste 3,91 frente a Sudeste 4,14), lo que confirma la relación a nivel geográfico.

12.4 P4 · Medios de pago y cuotas

Según el pago principal, la tarjeta de crédito cubre el 75,5 % de los pedidos, el boleto el 19,9 %, el voucher el 3,1 % y la tarjeta de débito el 1,5 %. El número de cuotas crece con el valor del pedido: el ticket con tarjeta pasa de R$ 100,83 en una cuota a R$ 154,80 en 2–6 cuotas, R$ 333,16 en 7–12 y R$ 417,71 en más de 12.

12.5 Decisiones que apoya la solución

Hallazgo	Usuario	Decisión recomendada
RJ concentra el 22,9 % de los retrasos con el 12,8 % de los pedidos; el Nordeste tiene la peor puntualidad.	Logística	Renegociar o sustituir transportistas en RJ y el Nordeste; recalibrar la fecha estimada por estado para que la promesa sea realista.
La puntualidad cae en Black Friday y en feb–mar 2018.	Logística y gerencia	Reservar capacidad de transporte antes de los picos y ampliar la fecha estimada en campañas de alta demanda.
Con 4 o más días de retraso, la mayoría de las reseñas es negativa.	Servicio al cliente	Alerta automática al superar la fecha estimada: contacto proactivo y cupón de compensación antes de que el cliente califique.
38 vendedores superan el 15 % de retrasos.	Gestión de vendedores	Plan de mejora con plazo; advertencia o suspensión si no mejora; reconocimiento a los vendedores de alto volumen con bajo retraso.
El 63 % del ingreso proviene de SP, RJ y MG, pero el ticket es mayor en el Norte y el Nordeste.	Comercial	Explorar crecimiento en el Nordeste solo después de corregir su logística: hoy el retraso destruye la satisfacción que el mayor ticket podría capitalizar.
Las cuotas acompañan a los pedidos de mayor valor.	Finanzas	Promociones de cuotas sin interés en categorías de ticket alto (watches_gifts, computers_accessories).

Tabla 20. De la información a la decisión.

13. Uso de Inteligencia Artificial

Se utilizó un asistente de IA generativa como apoyo para diseñar consultas, depurar código e interpretar resultados. Ninguna propuesta se aceptó sin ejecutarla y contrastarla con un cálculo independiente. La Tabla 21 resume los cinco usos más significativos; el detalle completo (prompt, propuesta, adaptación y comprobación) está en `docs/uso_ia.md`.

#	Problema	Prompt (resumen)	Propuesta de la IA	Qué se adaptó	Cómo se comprobó
1	OTD de 91,9 % (pandas) vs. 93,2 % (SQL).	¿Por qué difieren y cuál uso?	La fecha estimada no tiene hora; comparar fechas de calendario.	Comparación por DATE en la tabla de hechos; medidas solo para `delivered`.	Ambas versiones calculadas: la diferencia son exactamente 1 291 pedidos entregados el mismo día.

#	Problema	Prompt (resumen)	Propuesta de la IA	Qué se adaptó	Cómo se comprobó
2	La reseña se repite en cada ítem del pedido.	¿Cómo promediar sin sobrecontar?	CTE con `DISTINCT ON (order_id)`.	Aplicado a 4 funciones; en el ranking, por (vendedor, pedido).	9 767 pedidos con varios ítems; el CSAT por ítem resulta ≈ 0,07 menor. Validación 4c.
3	Ciudades en múltiples formas e inválidas.	Función pandas para normalizar ciudades.	NFKD + regex; imputar por prefijo postal.	Se añadió quitar la sigla final y el acento agudo.	7 casos revisados a mano; 611 → 581 ciudades; 0 sin ciudad.
4	Filtros que se reiniciaban al cambiar de página.	¿Cómo conservar filtros en multipágina?	Almacén propio en `st.session_state` + `on_change`.	Filtro de estado en cascada con la región.	Recorrido de las 4 páginas con filtros fijados; combinación sin datos.
5	Error de sintaxis al ejecutar validaciones desde Python.	¿Por qué falla un comentario?	Un `;` dentro de un comentario partía la consulta.	Se eliminan comentarios antes de dividir.	Pipeline completo sin errores; 14 consultas en el reporte.

Tabla 21. Usos significativos de IA.

Uso responsable. La elección del dataset, las preguntas de negocio, la decisión de mantener un único grano de ítem y la interpretación de los resultados son decisiones propias. Todo el código del repositorio puede ser explicado, ejecutado y modificado por la estudiante.

14. Conclusiones y limitaciones

14.1 Conclusiones

- Se construyó una solución de BI completa y reproducible con un solo comando, desde 9 archivos CSV hasta un dashboard de 4 páginas, con 14 validaciones automáticas que garantizan que los datos presentados coinciden con los originales.
- El modelo estrella con grano de ítem permite responder las cuatro preguntas de negocio; los indicadores de nivel pedido se obtienen sin sobreconteo.
- La hipótesis del Entregable 1 se confirma y se cuantifica: la puntualidad es el principal determinante de la satisfacción. Un retraso de 4 días o más reduce el CSAT a menos de la mitad.
- El problema logístico está localizado (Río de Janeiro, el Nordeste, los meses pico y un grupo reducido de vendedores), lo que permite acciones focalizadas en lugar de cambios generales.

14.2 Limitaciones

- El dataset no incluye la transportista ni costos operativos, por lo que el retraso solo puede atribuirse a estado, mes o vendedor.
- Los pedidos sin ítems (739 en el período) no forman parte de la tabla de hechos; la tasa de cancelación total requeriría una tabla de hechos a nivel de pedido.
- El análisis es descriptivo: la relación retraso–satisfacción es consistente y fuerte, pero no constituye por sí sola una prueba causal.
- Los datos terminan en agosto de 2018; el dashboard describe un período histórico y no se actualiza en línea.

15. Entrega, ejecución y video

Repositorio: https://github.com/MariaLlerenaHernandez/Brazilian-E-Commerce-Public-Dataset-by-Olist. El archivo `README.md` contiene las instrucciones completas. Resumen de ejecución:

Ejecución

```
pip install -r requirements.txt
docker compose up -d
python etl/run_pipeline.py
streamlit run app/Inicio.py
```

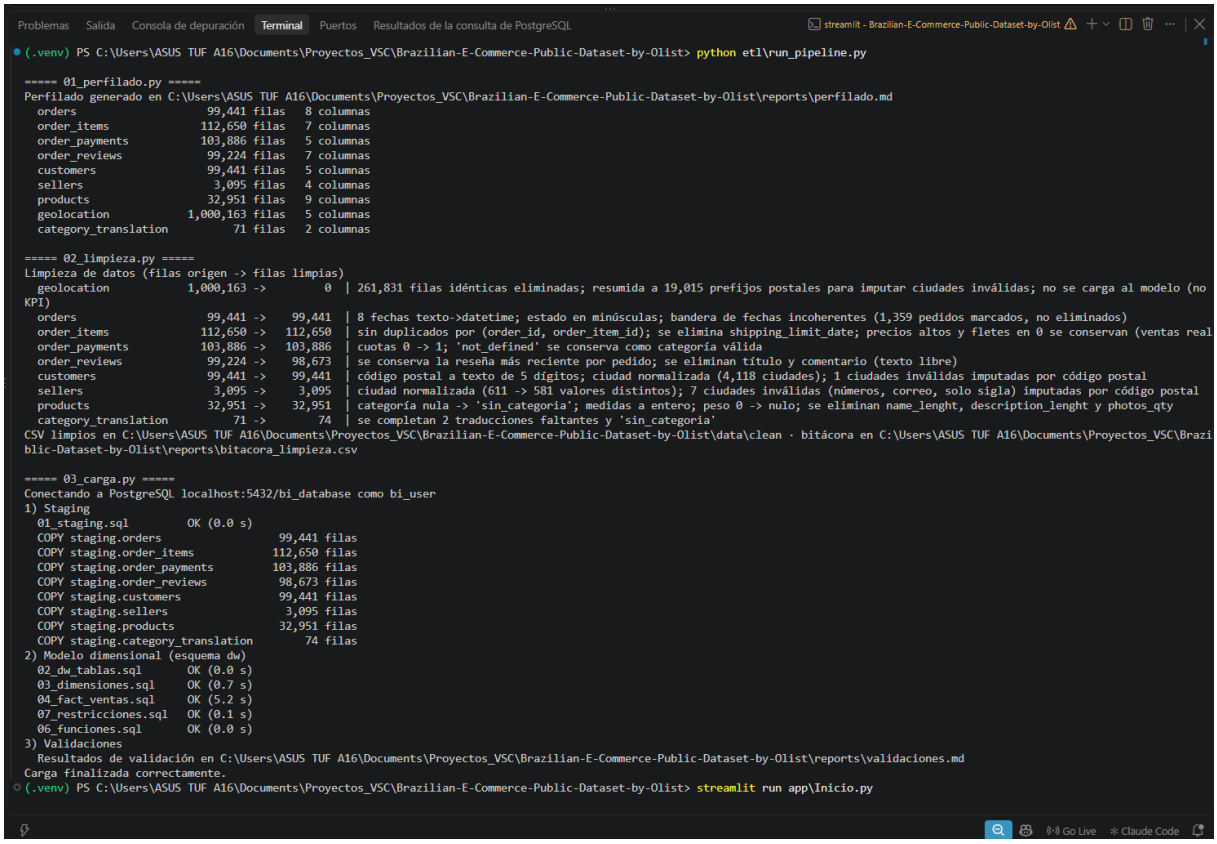
Componente exigido	Ubicación en el repositorio
Documento final del proyecto	docs/Entregable3_Informe_Final.docx
Enlace, descripción y justificación del dataset	Secciones 2.1–2.5 · data/README.md
Preguntas de negocio y KPI	Sección 3
Limpieza y transformación	Sección 4 · etl/01_perfilado.py, etl/02_limpieza.py · reports/
Modelo dimensional, diagrama estrella y granularidad	Sección 5 · docs/img/diagrama_estrella.png
Scripts SQL de creación	sql/01_staging.sql, sql/02_dw_tablas.sql, sql/07_restricciones.sql
Scripts o proceso de carga	etl/03_carga.py, sql/03_dimensiones.sql, sql/04_fact_ventas.sql
Consultas de validación	sql/05_validaciones.sql · reports/validaciones.md
Vistas y funciones SQL	sql/06_funciones.sql
Mockup	Sección 9 · docs/img/mockup_dashboard.png
Matriz de trazabilidad	Sección 10 · docs/matriz_trazabilidad.md
Código fuente Python y proyecto Streamlit	etl/ · app/
README con instrucciones de ejecución	README.md
Evidencias de uso de IA	Sección 13 · docs/uso_ia.md
Video individual de 3 a 5 minutos	Video de presentación_MariaLlerenaHernandez.mp4 · guion en docs/guion_video.md

Tabla 22. Lista de verificación de la entrega final.

Video. El guion (`docs/guion_video.md`) está distribuido en 4 min 30 s: problema y dataset (0:00), modelo y granularidad (0:30), PostgreSQL y `fn_kpis()` en Adminer (1:10), KPI y gráficos (1:50), cambio del filtro de región a Nordeste (2:40), resultado relevante y decisión (3:20) y uso de IA (4:05).

Anexo A. Evidencias de ejecución

Evidencia 1. Terminal con la salida de python etl/run_pipeline.py («Carga finalizada correctamente»).



```
Problemas  Salida  Consola de depuración  Terminal  Puertos  Resultados de la consulta de PostgreSQL
streamlit - Brazilian-E-Commerce-Public-Dataset-by-Olist

(.venv) PS C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazilian-E-Commerce-Public-Dataset-by-Olist> python etl\run_pipeline.py

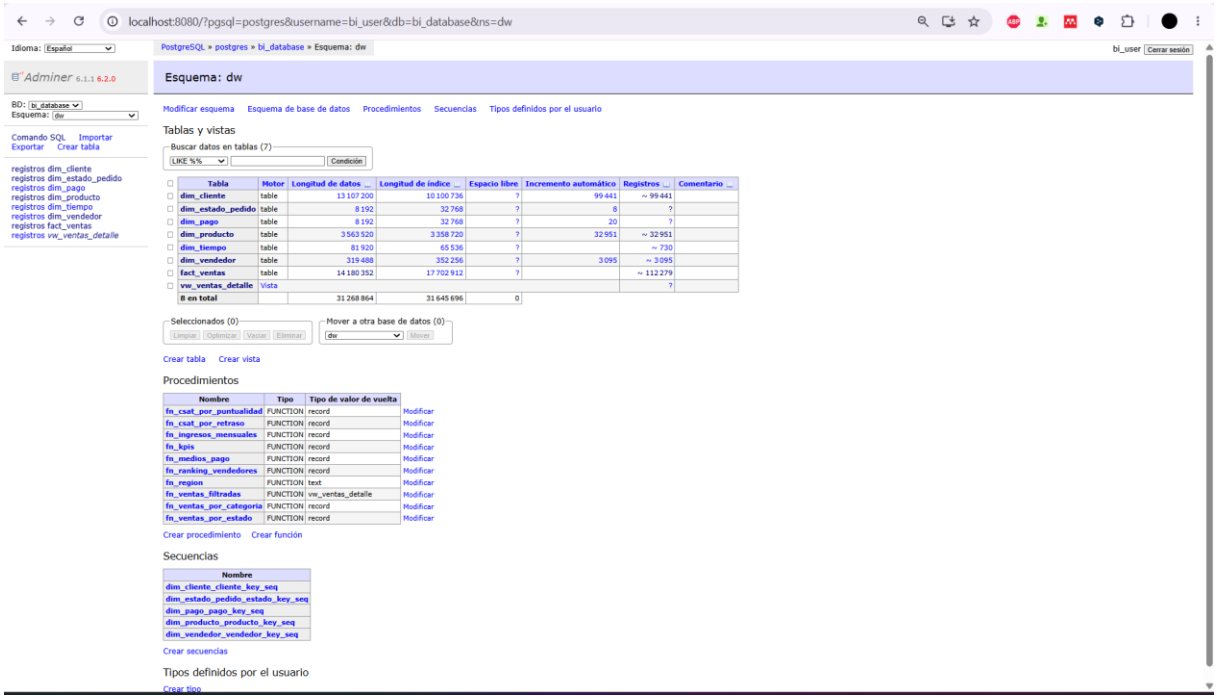
===== 01_perfilado.py =====
Perfilado generado en C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazilian-E-Commerce-Public-Dataset-by-Olist\reports\perfilado.md
  orders                    99,441 filas   8 columnas
  order_items              112,650 filas   7 columnas
  order_payments           103,886 filas   5 columnas
  order_reviews             99,224 filas   7 columnas
  customers                 99,441 filas   5 columnas
  sellers                    3,095 filas   4 columnas
  products                  32,951 filas   9 columnas
  geolocation            1,000,163 filas   5 columnas
  category_translation          71 filas   2 columnas

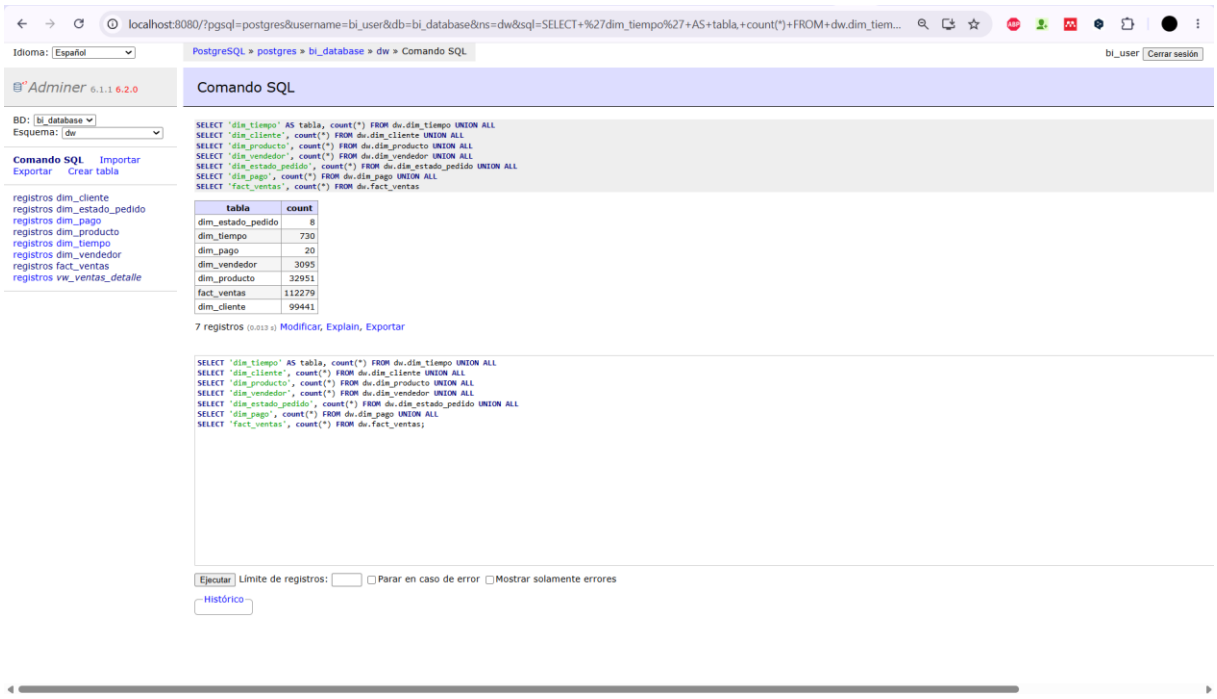
===== 02_limpieza.py =====
Limpieza de datos (filas origen -> filas limpias)
  geolocation           1,000,163 ->         0  | 261,831 filas idénticas eliminadas; resumida a 19,015 prefijos postales para imputar ciudades inválidas; no se carga al modelo (no
KPI)
  orders                   99,441 ->    99,441  | 8 fechas texto->datetime; estado en minúsculas; bandera de fechas incoherentes (1,359 pedidos marcados, no eliminados)
  order_items             112,650 ->   112,650  | sin duplicados por (order_id, order_item_id); se elimina shipping_limit_date; precios altos y fletes en 0 se conservan (ventas real
  order_payments          103,886 ->   103,886  | cuotas 0 -> 1; 'not_defined' se conserva como categoría válida
  order_reviews            99,224 ->    98,673  | se conserva la reseña más reciente por pedido; se eliminan título y comentario (texto libre)
  customers                99,441 ->    99,441  | código postal a texto de 5 dígitos; ciudad normalizada (4,118 ciudades); 1 ciudades inválidas imputadas por código postal
  sellers                   3,095 ->     3,095  | ciudad normalizada (611 -> 581 valores distintos); 7 ciudades inválidas (números, correo, solo sigla) imputadas por código postal
  products                 32,951 ->    32,951  | categoría nula -> 'sin_categoria'; medidas a entero; peso 0 -> nulo; se eliminan name_lenght, description_lenght y photos_qty
  category_translation         71 ->        74  | se completan 2 traducciones faltantes y 'sin_categoria'
CSV limpios en C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazilian-E-Commerce-Public-Dataset-by-Olist\data\clean · bitácora en C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazi
blic-Dataset-by-Olist\reports\bitacora_limpieza.csv

===== 03_carga.py =====
Conectando a PostgreSQL localhost:5432/bi_database como bi_user
1) Staging
  01_staging.sql         OK (0.0 s)
  COPY staging.orders                99,441 filas
  COPY staging.order_items          112,650 filas
  COPY staging.order_payments       103,886 filas
  COPY staging.order_reviews         98,673 filas
  COPY staging.customers             99,441 filas
  COPY staging.sellers                3,095 filas
  COPY staging.products              32,951 filas
  COPY staging.category_translation      74 filas
2) Modelo dimensional (esquema dw)
  02_dw_tablas.sql       OK (0.0 s)
  03_dimensiones.sql     OK (0.7 s)
  04_fact_ventas.sql     OK (5.2 s)
  07_restricciones.sql   OK (0.1 s)
  06_funciones.sql       OK (0.0 s)
3) Validaciones
  Resultados de validación en C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazilian-E-Commerce-Public-Dataset-by-Olist\reports\validaciones.md
Carga finalizada correctamente.
(.venv) PS C:\Users\ASUS TUF A16\Documents\Proyectos_VSC\Brazilian-E-Commerce-Public-Dataset-by-Olist> streamlit run app\Inicio.py

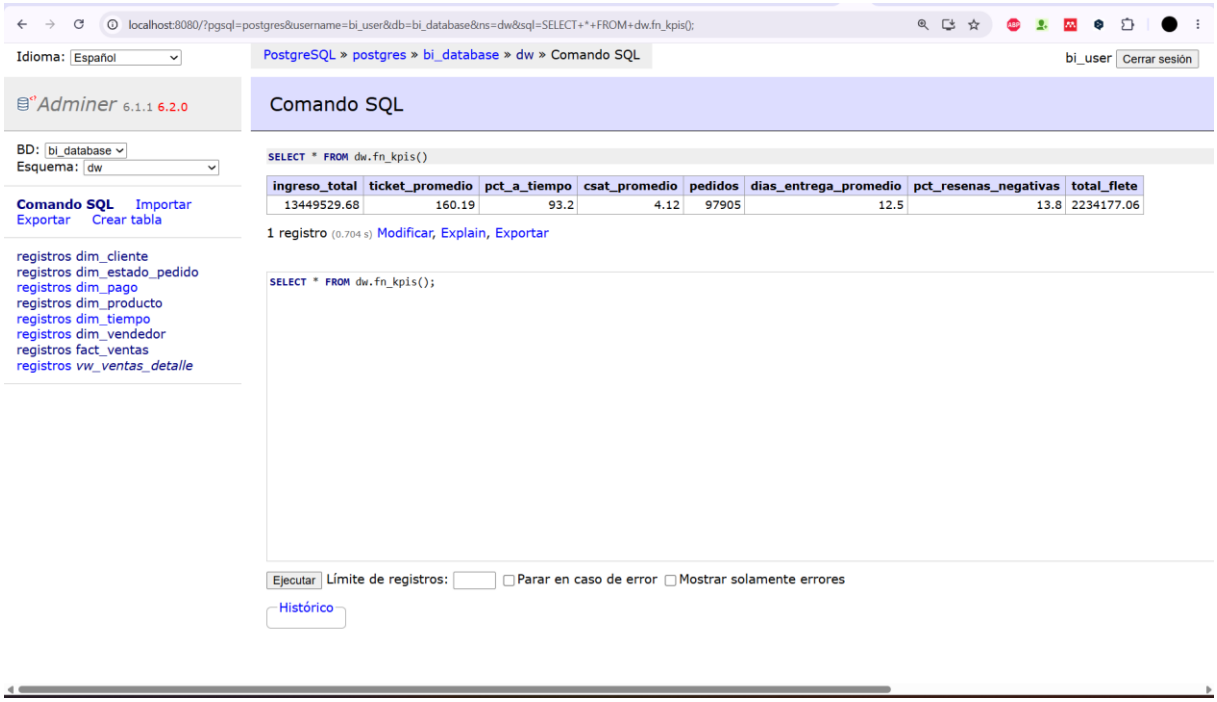
Go Live  Claude Code
```

Evidencia 2. Adminer: tablas del esquema dw con su número de filas.

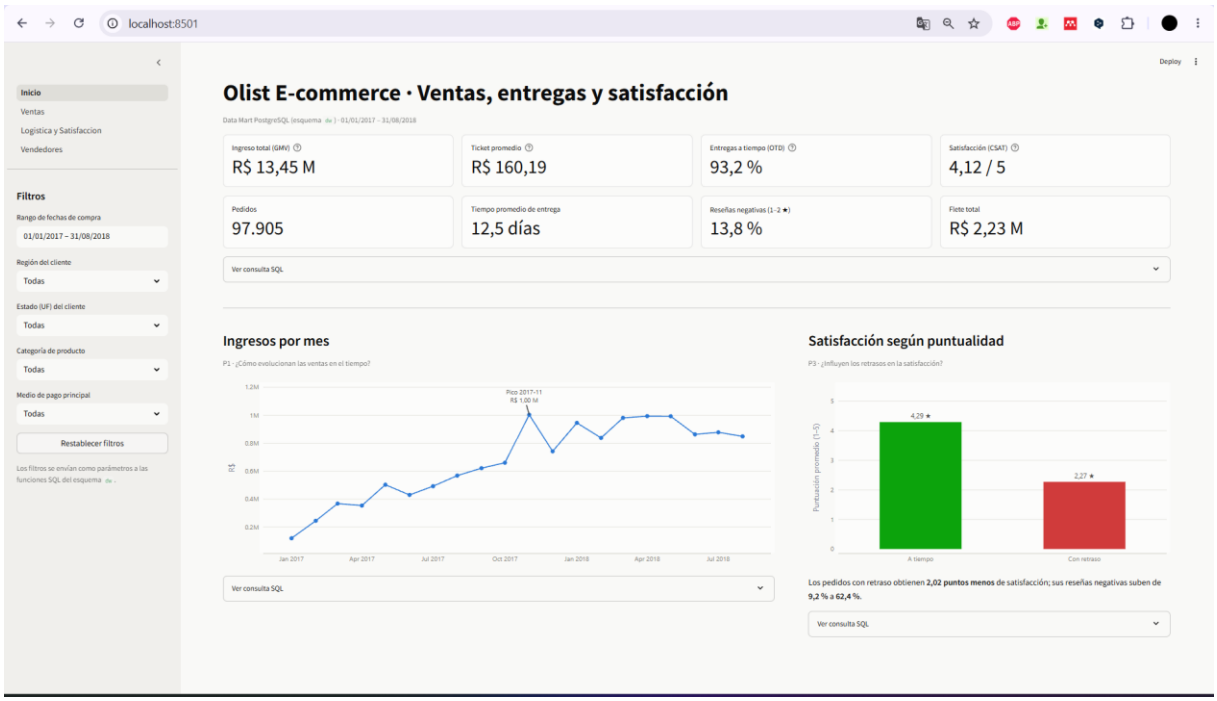




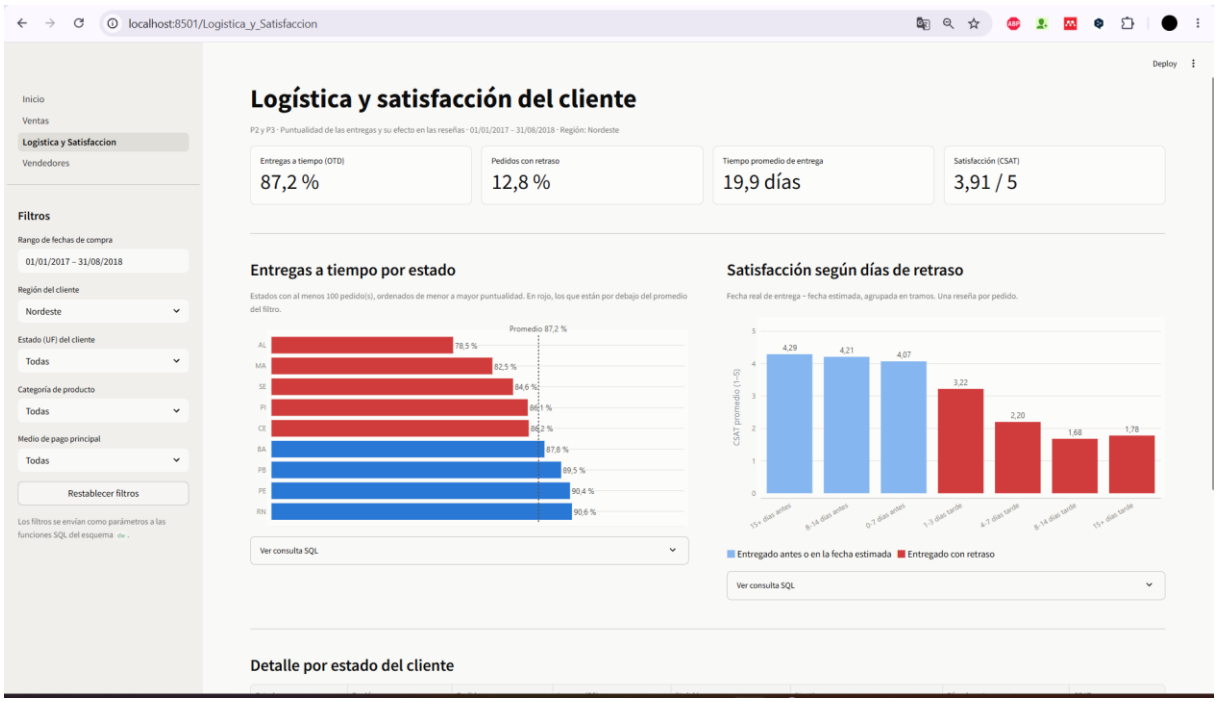
Evidencia 3. Adminer: resultado de SELECT * FROM dw.fn_kpis();

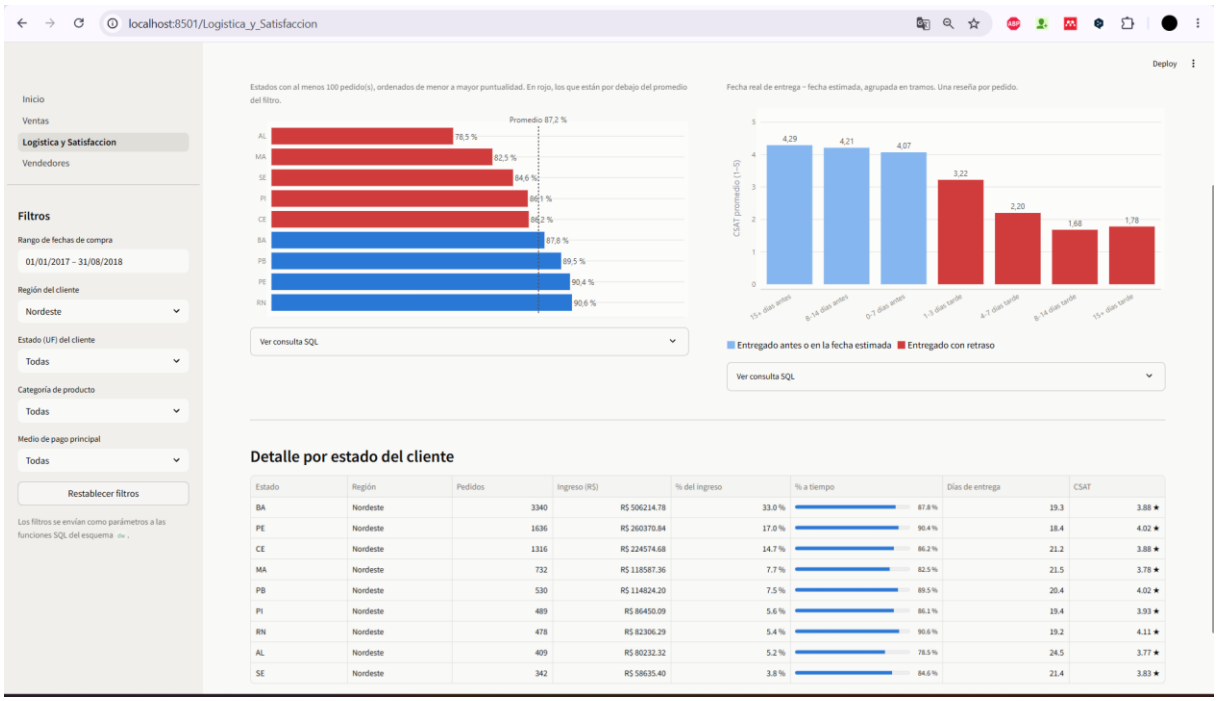


Evidencia 4. Streamlit · Inicio sin filtros (KPI y dos gráficos)

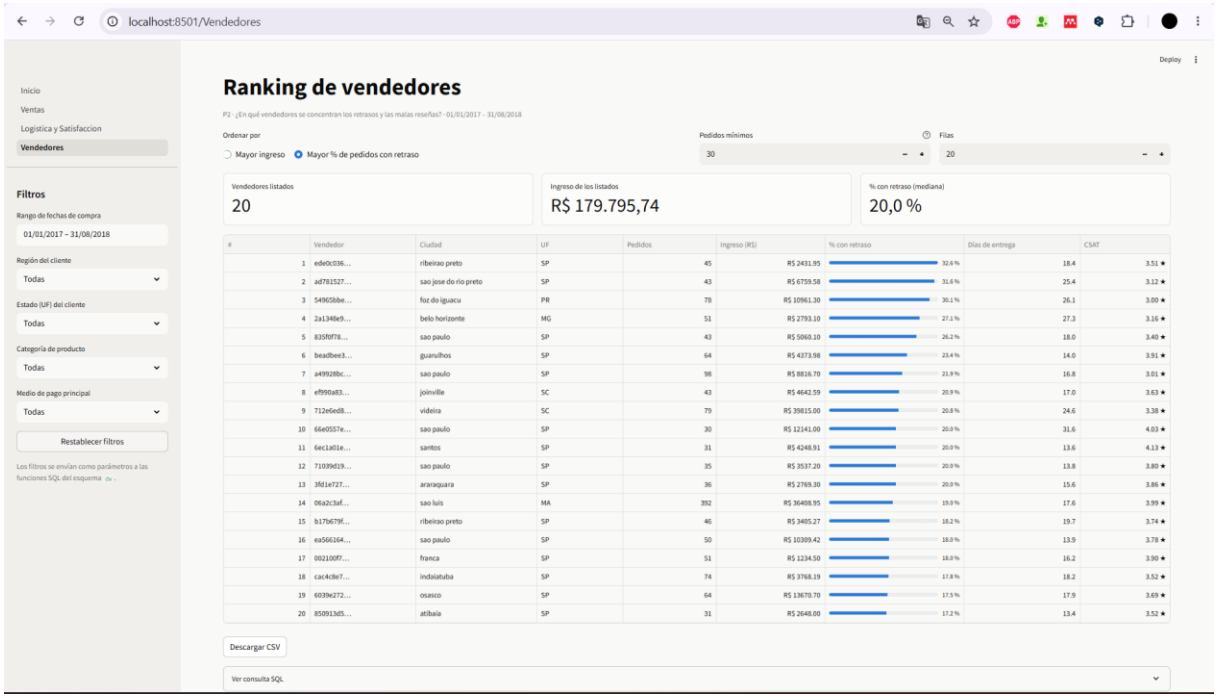


Evidencia 5. Streamlit · Logística y satisfacción con el filtro Región: Nordeste

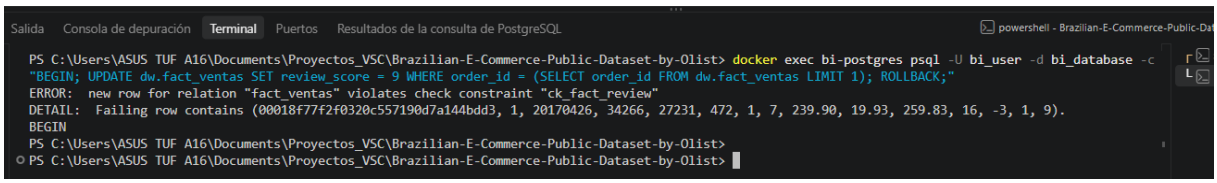




Evidencia 6. Streamlit · Vendedores ordenado por % con retraso (mínimo 30 pedidos)



Evidencia 7. Prueba de la restricción CHECK rechazando un review_score de 9



Referencias

Olist. (2018). *Brazilian E-Commerce Public Dataset by Olist* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Kimball, R. y Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3.ª ed.). Wiley.

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/

Streamlit Inc. (2025). *Streamlit documentation*. https://docs.streamlit.io

The pandas development team. (2024). *pandas documentation*. https://pandas.pydata.org/docs/